Finding entry by name	O(n)	O(n)	O(1) #	O(log n)	O(1) #	O(1) #
Eviction	O(n) §	O(1)	O(n)	O(log n)	O(1) #	O(log n) #

(#) Amortized time

7.5.4 Problems With LFU

Of course, no policy is always perfect, and even LFU has some CONs.

Most notably, if the cache runs for long, the turnaround time for older entries that are not popular anymore could be very long.

Let's make an example: if you have a cache with at most `n` entries, and the most frequently used one, call it `X`, has been requested `m` times before, at some point, not being accessed anymore (for example goes out of stock), then for a set of `n` new entries to cause the eviction of `X`, they will have to be requested `n*m` times at least.

Plugging in some numbers, if your cache holds a thousand elements and `X` was requested a thousand times, it will take at least one million accesses to a thousand brand new entries before `X` is evicted, after it becomes useless. Obviously, if more accesses are made to other entries already in the cache, then this number goes down, but on average it should be in that order of magnitude.

To solve this issue, some possible solutions are:

- Limiting the maximum value for the counter of an entry;